

Building a Reliable Data Transfer Protocol

1 Reliable Data Transfer over a Perfectly Reliable Channel: rdt1.0

We first consider the simplest case where the underlying channel is **perfectly reliable**. The protocol in this case is called **rdt1.0**.

Key Points of rdt1.0

- The channel never loses or corrupts packets.
- Sender and receiver each have a simple finite-state machine (FSM) with **one state**.
- No feedback from the receiver is required.
- The receiver can handle data as fast as the sender sends it.

1.1 Finite-State Machines (FSMs)

The sender and receiver are each modeled using a finite-state machine (FSM):

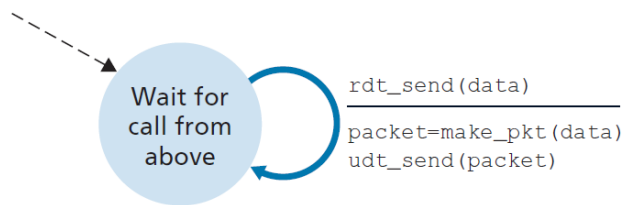
- **Sender FSM:** Accepts data from the upper layer via `rdt_send(data)`, creates a packet with `make_pkt(data)`, and sends it over the channel.
- **Receiver FSM:** Receives a packet via `rdt_rcv(packet)`, extracts the data using `extract(packet, data)`, and delivers it to the upper layer with `deliver_data(data)`.

1.2 Protocol Behavior

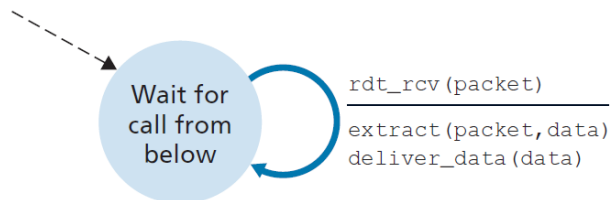
- There is **no distinction** between a unit of data and a packet.
- All packets flow from the sender to the receiver.
- Since the channel is perfectly reliable, no retransmissions, acknowledgments, or error handling are needed.
- The FSM always remains in the same state; transitions occur back to the same state on each event.

Summary of rdt1.0

- Simplest possible RDT protocol.
- Serves as a foundation for building more complex protocols for unreliable channels.
- Demonstrates the key ideas of the FSM-based protocol design.



a. rdt1.0: sending side



b. rdt1.0: receiving side

Figure 1: FSMs for rdt1.0: (a) Sender, (b) Receiver. Each FSM has only one state.